Institute for Complex Systems

JOHANNES KEPLER UNIVERSITÄT LINZ

Univ.-Prof. Dr. Daniel Große
DI Simon Reitinger & Katharina Ruep, M.Sc.

19. März 2026

Übung 2
zur Vorlesung Rechnerarchitektur

Abgabe bis **Donnerstag 26 März, 2026 10:15** via EPICS: `https://ep.ics.jku.at`.

RISC-V Programme müssen in der ICS Version von *Venus*[1] ausführbar sein. Zur Ausarbeitung der Übung darf nur der `RV32I` Befehlssatz mit der `M` Erweiterung verwendet werden. **Pseudobefehle sind (soweit nicht explizit erlaubt) nicht zu verwenden.** Die Lösungen sollten ausreichend kommentiert werden, im Speziellen sind Annahmen über die Registerbelegung der Variablen anzugeben. Beachte die RISC-V Calling Convention.[2]

[1] Online Simulator unter `https://venus.ics.jku.at/`.

[2] RISC-V Calling Convention 18.1 `https://riscv.org/wp-content/uploads/2024/12/riscv-calling.pdf`.

Aufgabe 1 – Disassembler (6 Punkte)

Übersetze das folgende binär kodierte RISC-V Programm zurück in RISC-V Assembler Code. (**inkl. Herleitung**). Es sollen Registernamen (z.B. ra, s0) verwendet werden und Adressen durch Labels ersetzt werden, welche auch im Programm ersichtlich sein sollen. Beschreibe die Funktionalität des Programms in Worten. Gibt es eine Änderung im Speicher, wenn ja welche?
Der Programm-Code (*.text*) startet bei Adresse `0x00000000`) und Speicher (*.data*) startet bei Adresse `0x10000000`.

Adresse	Befehl
0x00000000	10000417
0x00000004	00040413
0x00000008	00600493
0x0000000C	00A00913
0x00000010	00100E93
0x00000014	00048E63
0x00000018	00042383
0x0000001C	00440413
0x00000020	FFF48493
0x00000024	0123C463
0x00000028	027E8EB3
0x0000002C	FE9FF06F
0x00000030	10000E17
0x00000034	FE8E0E13
0x00000038	01DE2023

Adresse	Daten mit Offset			
	+00	+01	+02	+03
0x1000000C	..	..	..	..
0x10000018	0B	00	00	00
0x10000014	14	00	00	00
0x10000010	05	00	00	00
0x1000000C	0C	00	00	00
0x10000008	07	00	00	00
0x10000004	FF	FF	FF	FF
0x10000000	03	00	00	00
	LSB			MSB

Aufgabe 2 – Clamped Sum (10 Punkte)

Das folgende C-Program iteriert durch das Array `array`, begrenzt die Ergebnisse zwischen den übergebenen Werten `lo` und `hi` in einer separaten Funktion, addiert die Werte zur Summe `sum` und gibt diese zurück. Der Pseudobefehl `la` kann verwendet werden. Gib die Ergebnisse für clamped_sum$(5, 10)$ und clamped_sum$(15, 50)$ an.

```
#include <stdio.h>

int array[] = {4, 6, 8, 2, 88, 9, 37, 200, 84, 18};
int size = 10;

int clamp_int(int x, int lo, int hi) {
  if (x < lo) return lo;
  if (x > hi) return hi;
  return x;
}

int clamped_sum(int lo, int hi) {
  int sum = 0;
  for (int i = 0; i < size; i++) {
    int v = array[i];
    v = clamp_int(v, lo, hi);
    sum += v;
  }
  return sum;
}

int main() {
  int lo = 5;
  int hi = 10;
  int sum1 = clamped_sum(lo, hi);

  lo = 15;
  hi = 50;
  int sum2 = clamped_sum(lo, hi);

  printf("%d", sum1);
  printf("\n");
  printf("%d", sum2);
}
```

Benutze das angefügte Assembler Gerüst: *clamped_sum_frame.s*

Aufgabe 3 – Collatz-Schrittzahl (8 Punkte)

Die folgende RISC-V Assembler-Funktion zählt für eine positive Eingabe n die Anzahl der Schritte, um mittels Collatz-Iteration den Wert 1 zu erreichen.
Es gilt:

$$steps(n) = \begin{cases} 0, & n = 1 \\ 1 + steps(\frac{n}{2}), & n \text{ gerade} \\ 1 + steps(3n+1), & n \text{ ungerade} \end{cases}$$

Führe die gegebene Funktion in eine rekursive Funktion über. `main` soll dabei erhalten bleiben.
Gib die Ergebnisse für collatz_recu(42) und collatz_recu(1337) an.

```
.globl main

.text

collatz_iter: addi t3, zero, 0  # steps = 0
  addi t0, zero, 1
  beq a0, t0, c_done  # if (n == 1) return 0

c_loop: andi t1, a0, 1
  bnez t1, c_odd    # if (odd)
  srli a0, a0, 1    # n = n / 2
  addi t3, t3, 1    # steps++
  bne a0, t0, c_loop
  jal zero, c_done

c_odd: addi t2, zero, 3
  mul t2, t2, a0    # 3n
  addi a0, t2, 1    # n = 3n + 1
  addi t3, t3, 1    # steps++
  bne a0, t0, c_loop

c_done: addi a0, t3, 0    # return steps
  jalr zero, ra, 0

main:addi a0, zero, 42    # n = 42
  jal ra, collatz_iter
  addi a1, a0, 0    # s0 = collatz(42)
  addi a0, zero, 1  # ecall print_int
  ecall

  addi a0, zero, 11   # ecall print_char
  addi a1, zero, '\n'
  ecall

  addi a0, zero, 1337 # n = 1337
  jal ra, collatz_iter
  addi a1, a0, 0    # s1 = collatz(1337)
  addi a0, zero, 1  # ecall print_int
  ecall
```